SE3040 – Application Frameworks

BSc (Hons) in Information Technology Specialized in Software Engineering

Year 03 Semester 01

Testing Report

SE Group Number – SE-62

	Registration Number	Student Name	Student E-mail Address	Contact Number
1.	IT23668690	EDIRISINGHE D V M	it23668690@my.sliit.lk	0752903525
2.	IT23587038	PERERA J.A.A.H.D	it23587038@my.sliit.lk	0717466262
3.	IT23733862	RATHNASEKARA R K A S T	it23733862@my.sliit.lk	0711396048
4.	IT23591134	MUTHUNILMI T.G.K	it23591134@my.sliit.lk	0761620836

Table of content

01.How to Run Unit Tests 3
1.1. Unit Testing 3
1.2. Executing Unit Tests 3
1.3. Unit Testing Scenarios 4
1.3.1. Unit Testing - Login Module 4
1.3.2. Unit Testing - Appliance Module 6
1.3.3. Unit Testing - Waste Tracker Module 7
1.3.4. Unit Testing - Marketplace Module 8
1.3.5. Unit Testing - Recycle center Module 9
1.4. Outcome of Unit Testing 10
2. Integration testing setup and execution 11
2.1. Integration Testing 11
2.2. Group Testing Approach 11
2.3. Integration Test Setup and Execution Process 11
2.4. How Integration Testing Was Applied in This Project 12
2.4.1. Appliance CRUD (Energy Module) 13
2.4.2. Tariff Management CRUD (Energy Module) 13
2.4.3. Waste Log CRUD 15
2.4.4. Marketplace CRUD 15
3. Performance testing setup and execution 18
3.1. Performance Testing 18
3.2. Setup 18
3.3. Execution 18
3.4. Results and Analysis 20
3.5. Conclusion 20
4. Testing Environment Configuration Details 21
5. Overall Conclusion 22

01.How to Run Unit Tests

1.1. Unit Testing

Unit testing is the process of testing individual components or small parts of an application in isolation to ensure they function correctly. It focuses on verifying that each function or feature behaves as expected before integrating it with other parts of the system. In this project, unit-level testing was performed by validating individual CRUD-related features separately.

Unit testing was carried out as a group activity by dividing the CRUD functionalities among the four team members. Each member was responsible for testing a specific feature independently. This approach ensured that each module was verified in isolation before integrating all components into the full system.

1.2. Executing Unit Tests

Unit-related tests were executed using Playwright. To run the tests, both the frontend and backend servers must be started first. After that, the Playwright test command can be executed from the client directory. For example:

```
npx playwright test tests/testfilename.spec.js --headed
```

The --headed option allows the browser to open during execution so the test flow can be visually observed. Playwright generates a detailed report showing each step, execution time, and the final status of the test.

1.3. Unit Testing Scenarios

The following components were tested as part of unit testing:

1.3.1. Unit Testing - Login Module

The objective of this test was to check whether the login form properly validates user input before submitting the data. The main focus was to verify that invalid email formats are detected correctly.

Testing Process: Using Playwright, an invalid email (without a proper @ and domain) was entered along with a password. Then the login form was submitted. The system's response was observed to check whether the invalid input was rejected. The test also checked if the email input field remained focused after submission.

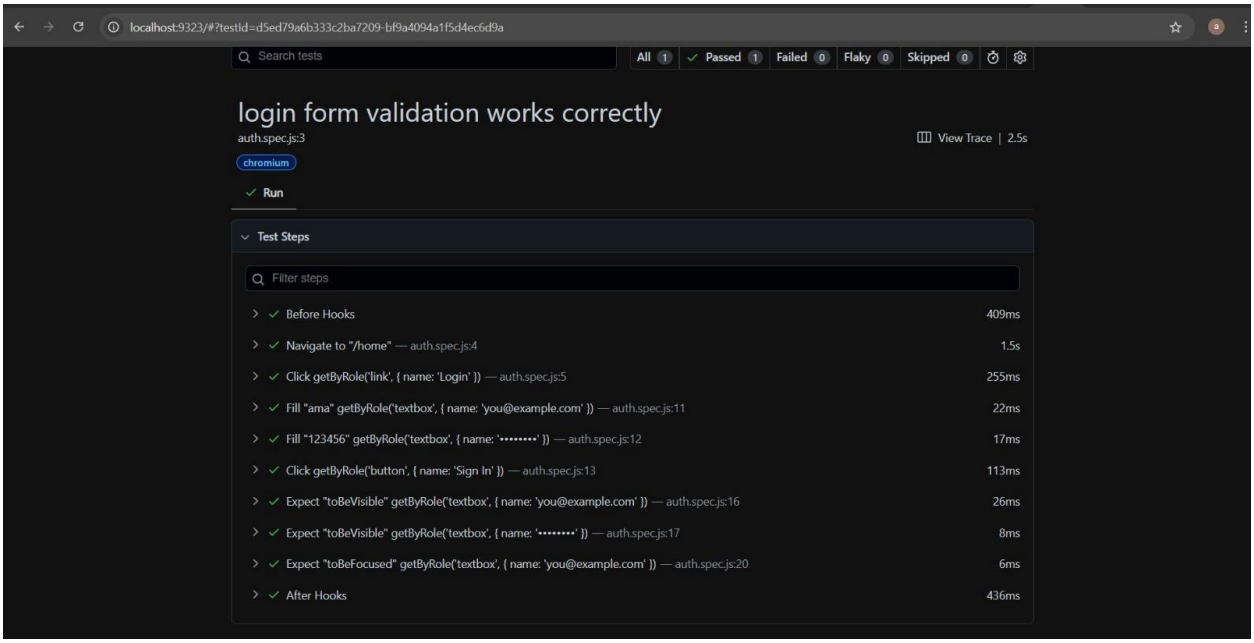
Results:

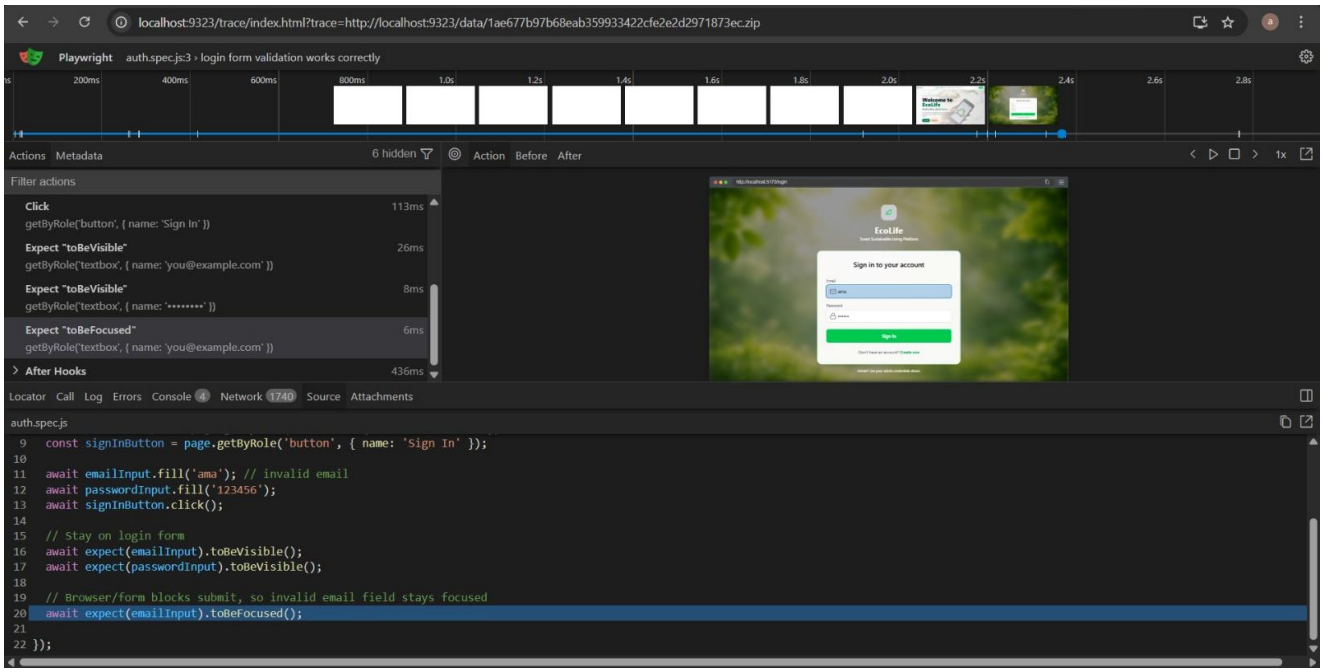
• Validation: The system correctly detected the invalid email format and did not allow submission.

• Focus Behavior: The email input field remained focused, allowing the user to fix the mistake.

• Integrity: The login request was blocked, preventing incorrect data from being sent.

• Conclusion: The login form validation works correctly and prevents invalid email formats from being submitted.


localhost:9323/#?testId=d5ed79a6b333c2ba7209-bf9a4094a1f5d4ec6d9a
Search tests
All 1
Passed 1
Failed 0
Flaky 0
Skipped 0
login form validation works correctly
auth.spec.js:3
View Trace | 2.5s
chromium
Run
Test Steps
Filter steps
Before Hooks 409ms
Navigate to "/home" — auth.spec.js:4 1.5s
Click getByRole('link', { name: 'Login' }) — auth.spec.js:5 255ms
Fill "ama" getByRole('textbox', { name: 'you@example.com' }) — auth.spec.js:11 22ms
Fill "123456" getByRole('textbox', { name: '••••••••' }) — auth.spec.js:12 17ms
Click getByRole('button', { name: 'Sign In' }) — auth.spec.js:13 113ms
Expect "toBeVisible" getByRole('textbox', { name: 'you@example.com' }) — auth.spec.js:16 26ms
Expect "toBeVisible" getByRole('textbox', { name: '••••••••' }) — auth.spec.js:17 8ms
Expect "toBeFocused" getByRole('textbox', { name: 'you@example.com' }) — auth.spec.js:20 6ms
After Hooks 436ms


localhost:9323/trace/index.html?trace=http://localhost:9323/data/1ae677b97b68eab359933422cfe2e2d2971873ec.zip
Playwright auth.spec.js:3 › login form validation works correctly
Actions Metadata 6 hidden
Action Before After
Filter actions
Click 113ms
getByRole('button', { name: 'Sign In' })
Expect "toBeVisible" 26ms
getByRole('textbox', { name: 'you@example.com' })
Expect "toBeVisible" 8ms
getByRole('textbox', { name: '••••••••' })
Expect "toBeFocused" 6ms
getByRole('textbox', { name: 'you@example.com' })
After Hooks 436ms
Locator Call Log Errors Console 4 Network 1740 Source Attachments
auth.spec.js
const signInButton = page.getByRole('button', { name: 'Sign In' });
await emailInput.fill('ama'); // invalid email
await passwordInput.fill('123456');
await signInButton.click();
// Stay on login form
await expect(emailInput).toBeVisible();
await expect(passwordInput).toBeVisible();
// Browser/form blocks submit, so invalid email field stays focused
await expect(emailInput).toBeFocused();
});

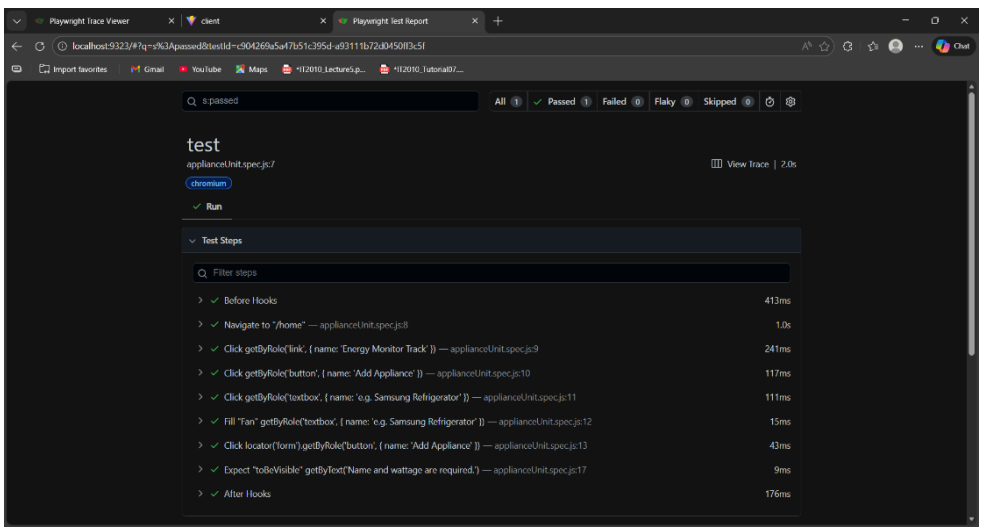
1.3.2. Unit Testing - Appliance Module

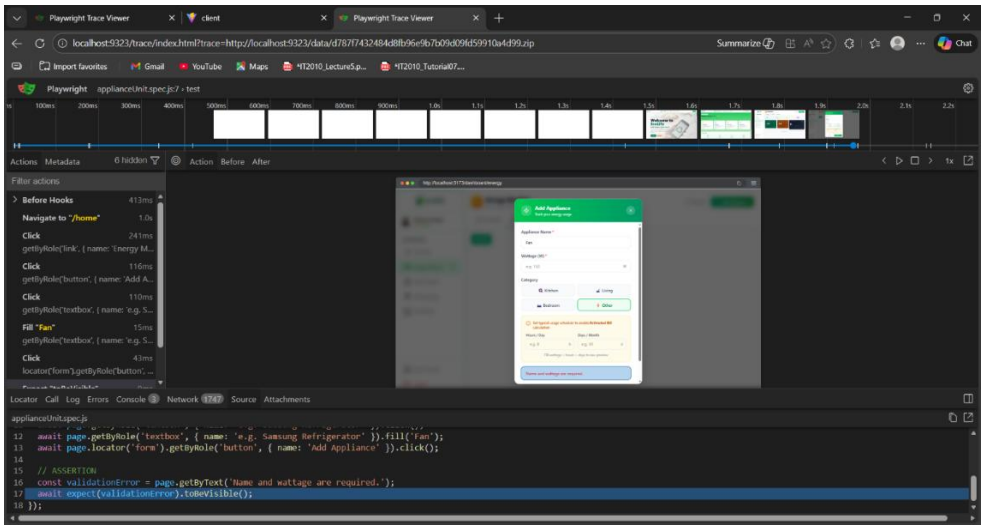
The objective of unit testing the Appliance Module was to ensure that the individual functions, specifically the **Appliance Creation Form**, handle data validation and business logic correctly before interacting with the database.

Testing Process: Using Playwright, the test entered only the appliance name and attempted to submit the form without providing the wattage. The system's ability to trigger a validation alert was then evaluated.

Results:

- **Validation:** The system correctly identified the missing input and displayed the error: *"Name and wattage are required."*
- **Integrity:** The submission was successfully blocked, ensuring that incomplete records cannot be saved to the database.
- **Conclusion:** The mandatory field validation logic for the Appliance module is functioning correctly and ensures data completeness.



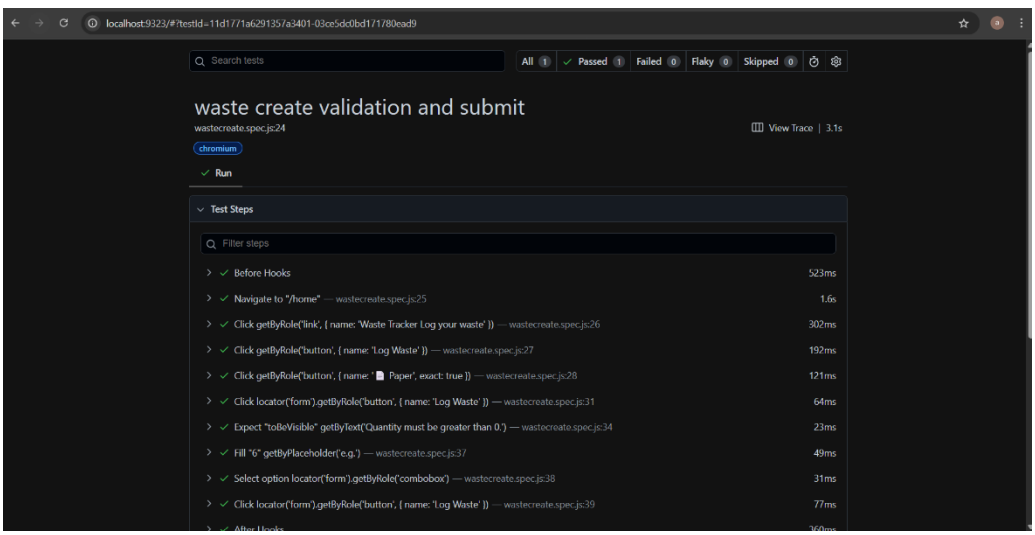


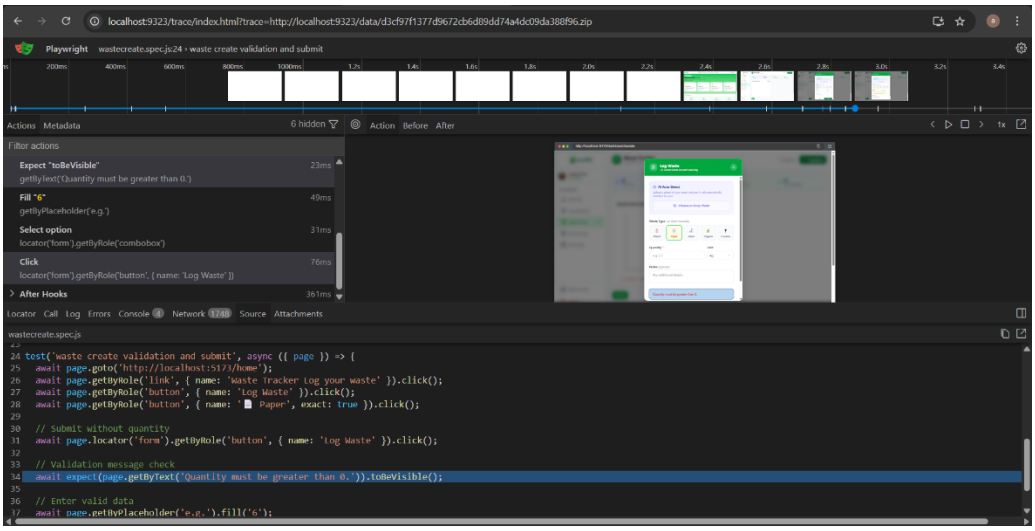
1.3.3. Unit Testing - Waste Tracker Module

The objective of unit testing the Waste Tracker Module was to ensure that the Waste Log creation form correctly handles required field validation and submission behavior before processing data further in the system.

Testing Process: Using Playwright, the test opened the Waste Log form, selected a waste type, and attempted to submit the form without entering the required quantity value. The system's validation behavior was then observed. After verifying the validation response, a valid quantity was entered and the form was submitted again to confirm successful creation.

Results:
• Validation: The system correctly detected the missing required input and displayed the validation error message: "Quantity must be greater than 0."
• Integrity: The invalid submission was blocked, preventing incomplete or incorrect data from being processed.
• Functionality: After entering a valid quantity, the form was submitted successfully.
• Conclusion: The Waste Tracker module correctly enforces required field validation and allows the create operation only when valid input is provided.





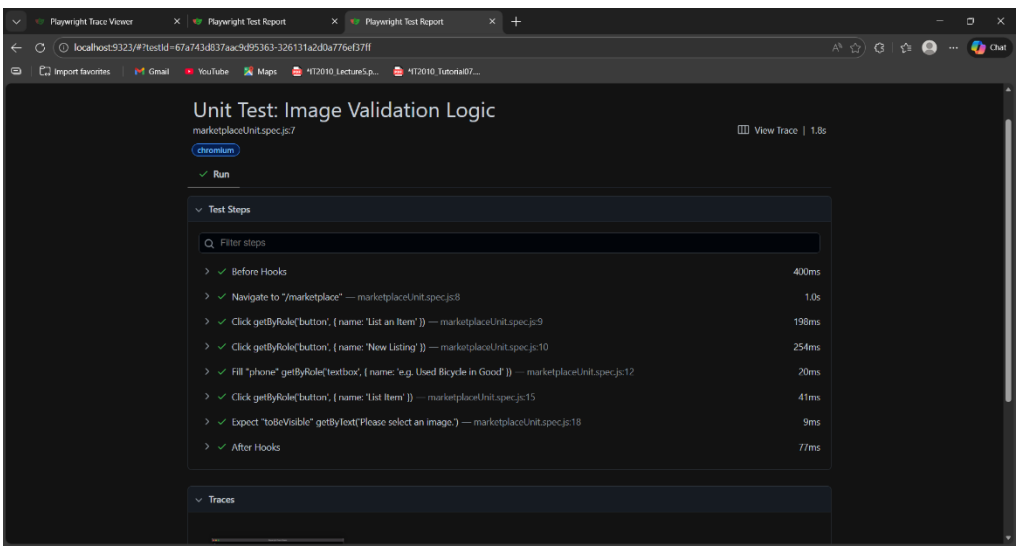
1.3.4. Unit Testing - Marketplace Module

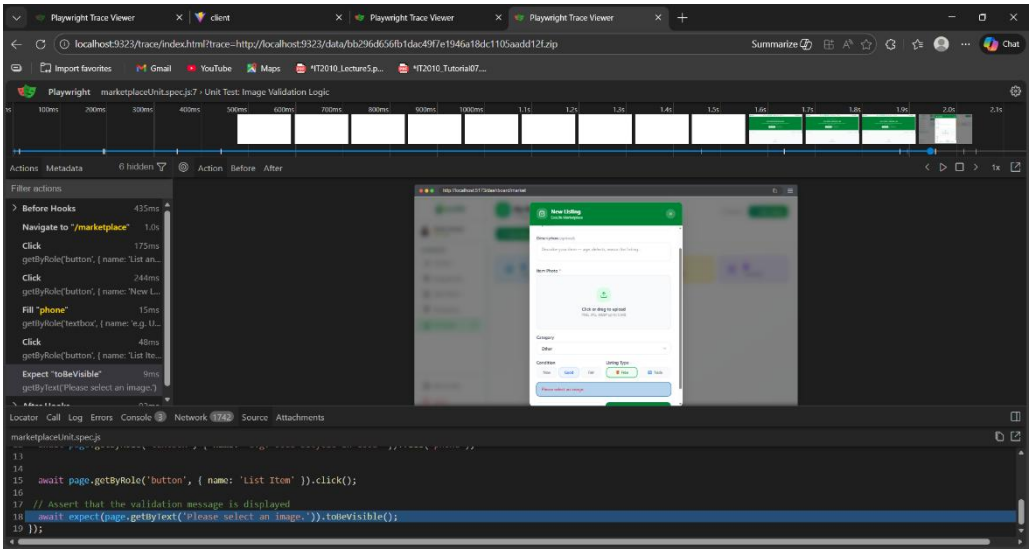
The unit test for the Marketplace module focused on verifying the Image Upload Validation logic. The objective was to ensure the application enforces business rules by preventing incomplete listings from being published.

Testing Process: Using Playwright, the test simulated a user entering a product title but intentionally skipping the image upload. Upon clicking the "List Item" button, the system's ability to intercept the invalid submission and provide immediate feedback was evaluated.

Results:

- **Validation Accuracy:** The system correctly identified the missing image and displayed the error message: *"Please select an image."*
- **Submission Prevention:** The form was successfully blocked from proceeding to the backend, ensuring data integrity.
- **Conclusion:** The test confirmed that the mandatory field logic for the Marketplace listing feature is robust and functions as expected.





1.3.5. Unit Testing - Recycle center Module

The objective of unit testing the Recycle Center Request Module was to ensure that the request submission process works correctly, including form handling and user feedback after submission.

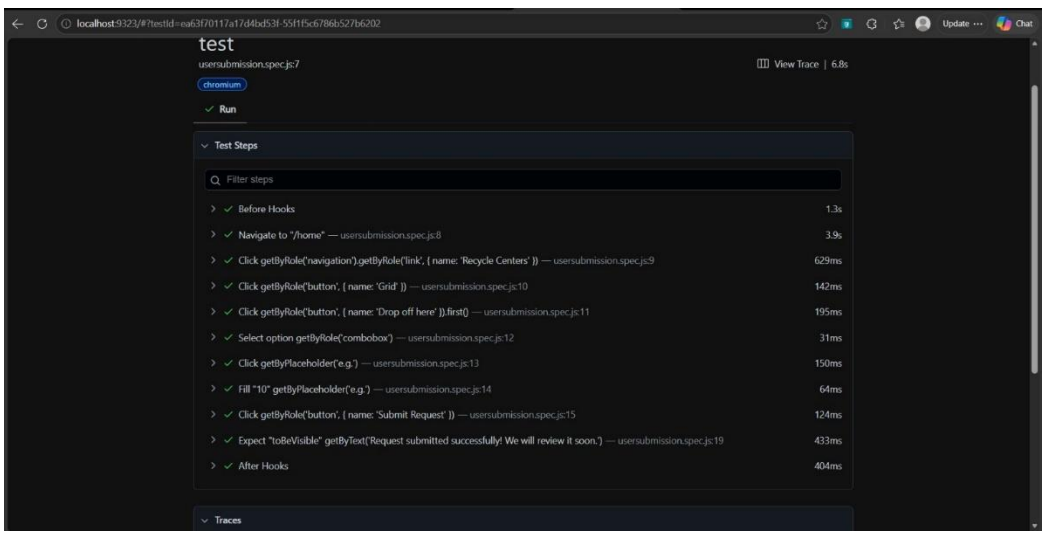
Testing Process: Using Playwright, the test logged in with a stored session, navigated to the Recycle Centers page, selected Grid view, and initiated a drop-off request. The user selected a waste type , entered a valid quantity, and submitted the form. The system's response and success notification were evaluated.

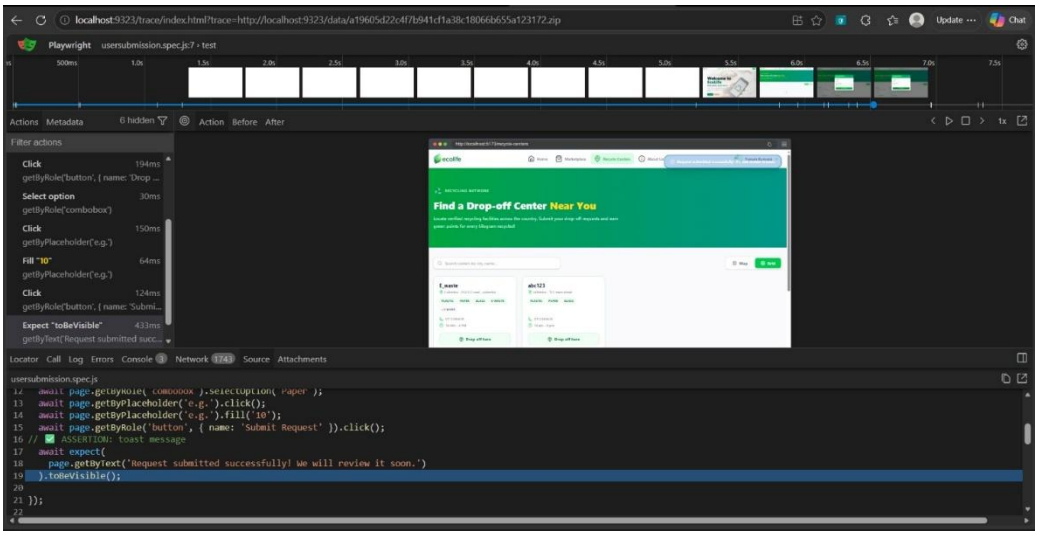
Results:

Functionality: The request was submitted successfully with valid inputs.

Feedback: A toast message was displayed: "Request submitted successfully! We will review it soon."

Conclusion: The module correctly handles request submission and provides proper user feedback.





1.4. Outcome of Unit Testing

Through these tests, individual features were validated to ensure correct behavior before integrating them into the full system. This helped reduce errors in later stages and improved overall application stability.

2. Integration testing setup and execution

2.1. Integration Testing

Integration testing is used to verify that different parts of the application work together correctly. Unlike unit testing, which focuses on individual features in isolation, integration testing checks whether connected modules such as the frontend, backend controllers, services, API routes, authentication, and the MongoDB database interact properly as a complete workflow.

In this project, integration testing was carried out to ensure that data flows correctly between the user interface, API endpoints, backend logic, and the database. It was also used to verify that requests and responses were handled correctly, including validation and error scenarios.

connected, including error scenarios.

2.2. Group Testing Approach

Integration testing was performed as a group activity. Each team member tested the full CRUD workflow of their assigned module using Playwright. These tests started from login and continued through the relevant create, read, update, and delete operations. This approach helped confirm that the connected parts of the system worked together correctly in real usage scenarios.

2.3. Integration Test Setup and Execution Process

Integration tests were executed using Playwright by simulating complete user flows through the application. Before running the tests, both the frontend and backend servers were started, and the MongoDB database connection was configured. Authentication was handled through a saved login state so that protected routes could be accessed during test execution.

To run an integration test, the relevant Playwright test file was executed from the client directory. For example:

```
npx playwright test tests/wasteCrud.spec.js --headed
```

The test then interacted with the application through the browser, starting from login and continuing through the required CRUD actions. During execution, Playwright validated whether the frontend sent requests correctly, whether the backend processed them properly, and whether the resulting changes were reflected in the user interface. This confirmed the integration between the frontend, backend API, application logic, and MongoDB database.

2.4. How Integration Testing Was Applied in This Project

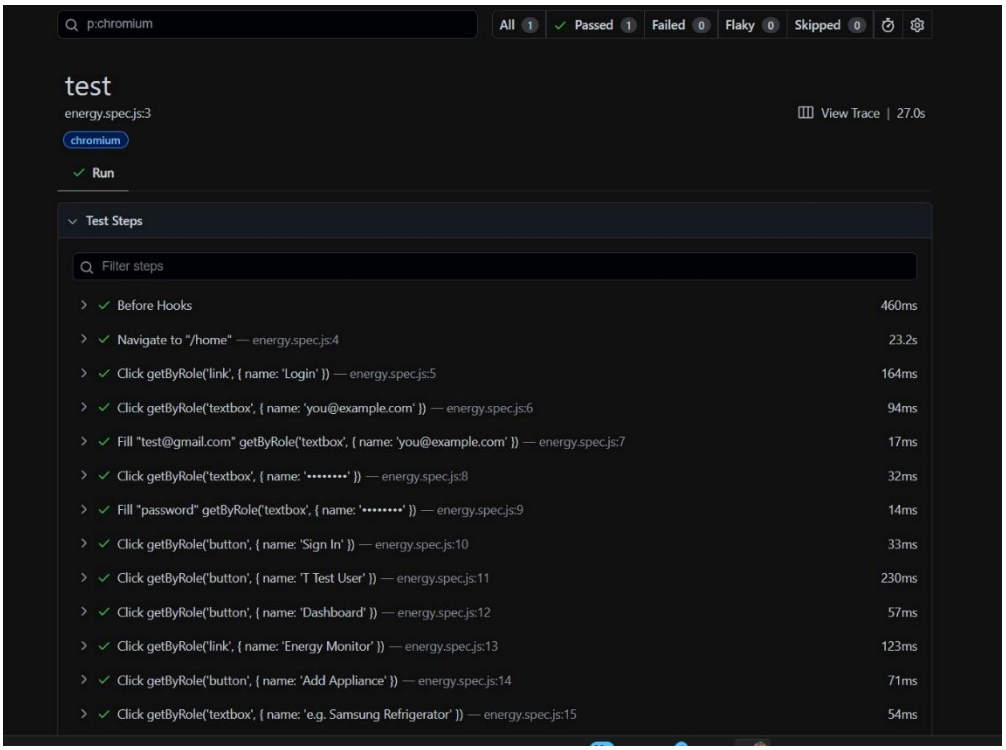
In this project, integration testing covered complete feature workflows rather than isolated actions. The tests checked whether:

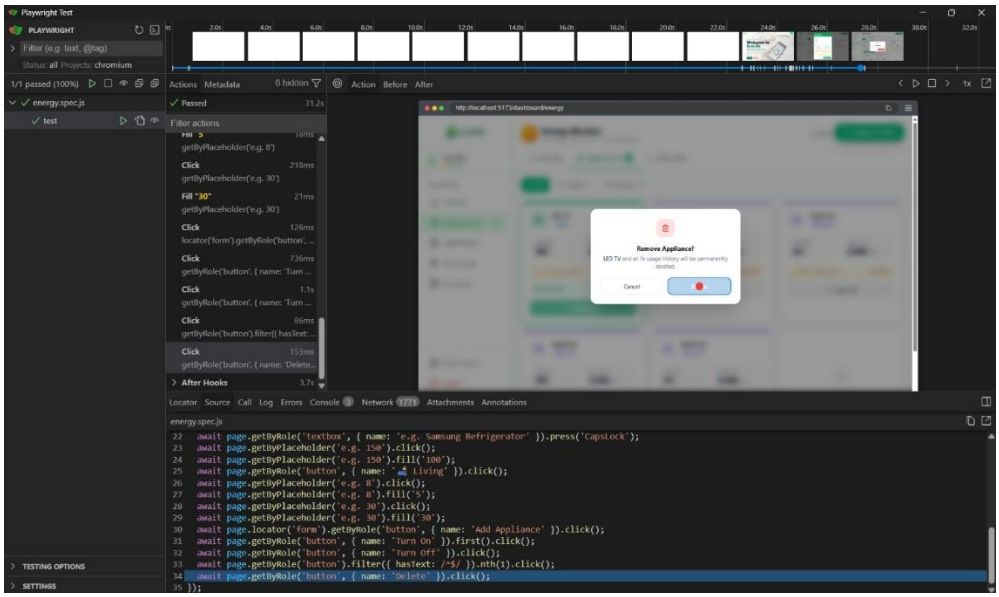
- the user could log in successfully,
- protected pages were accessible after authentication,
- forms submitted data correctly,
- backend endpoints processed requests and returned the correct responses,
- database changes were reflected in the user interface,
- update and delete actions worked as expected,
- and error or validation scenarios were handled properly.

This ensured that multiple layers of the system functioned together as a connected application.

2.4.1. Appliance CRUD (Energy Module)

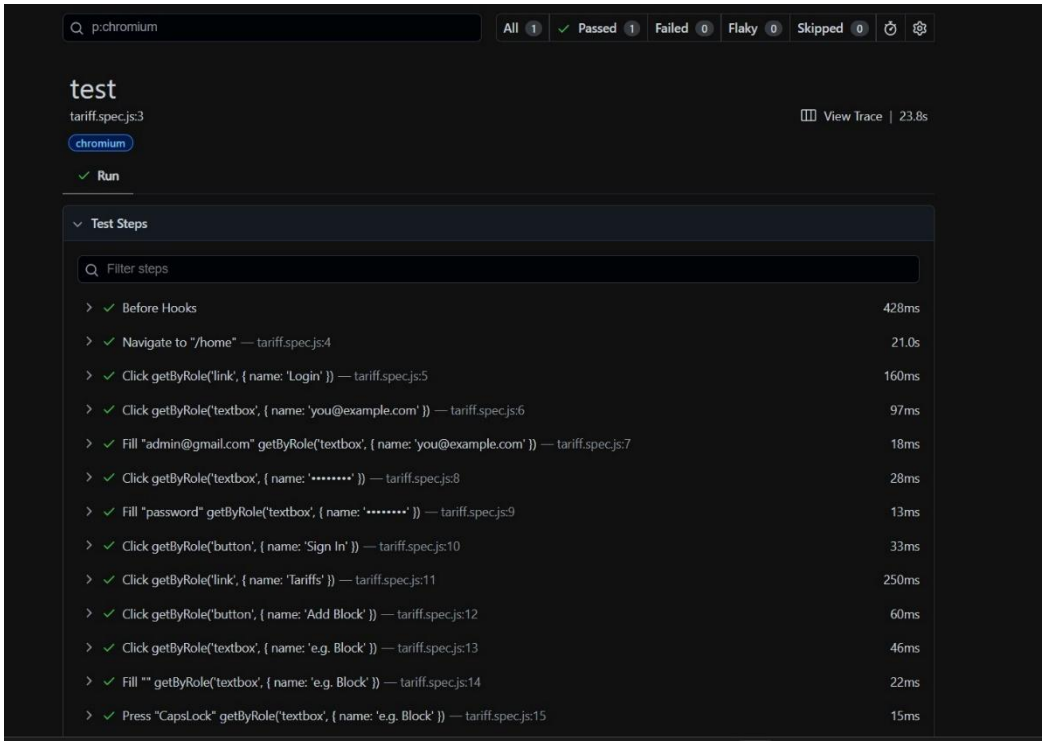
ntegration testing for the Energy Monitor module was carried out by validating the complete workflow from login to appliance management. The test confirmed that users could successfully perform CRUD operations, and that the frontend, backend, and database interacted correctly. The Playwright report showed all steps executed successfully, indicating proper integration of system components.

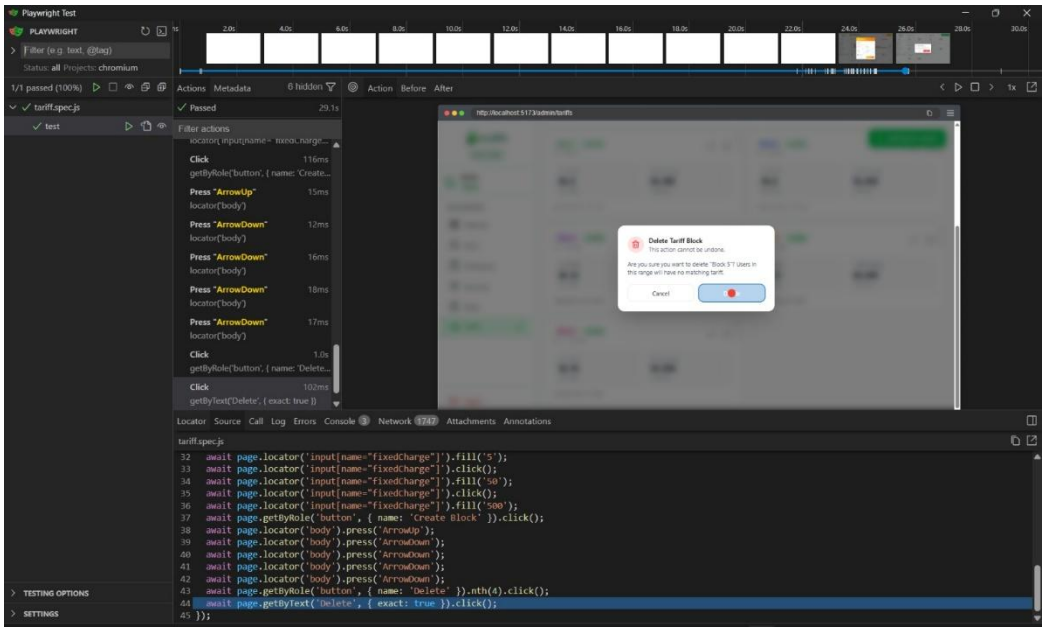




2.4.2. Tariff Management CRUD (Energy Module)

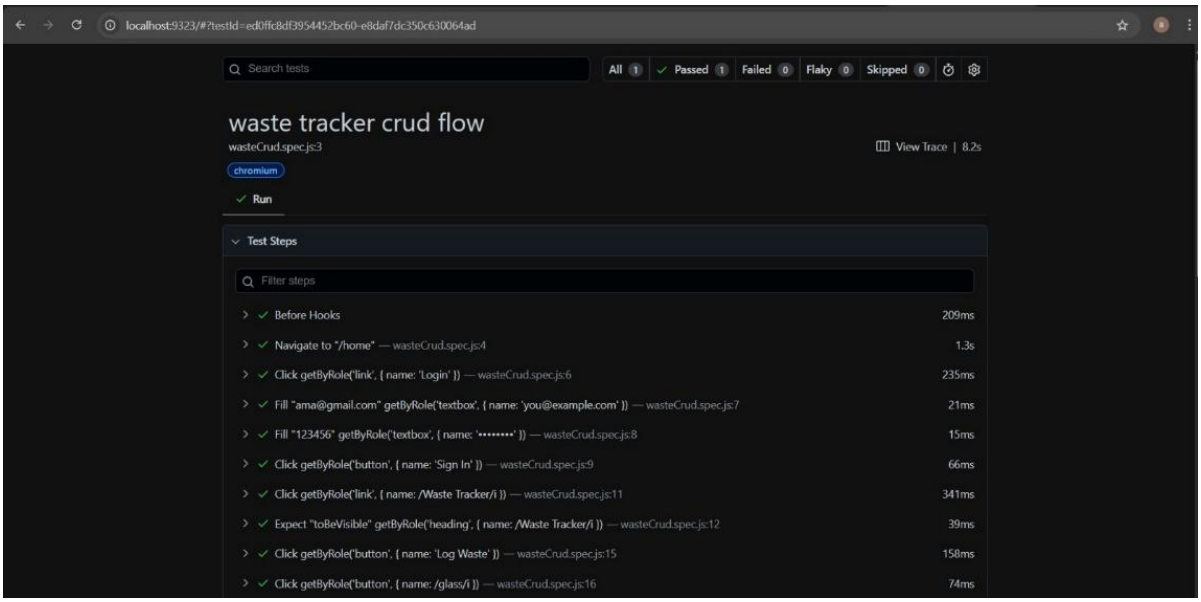
Integration testing for the Tariff Management module was carried out by validating the full workflow from login to tariff block management. The test confirmed that CRUD operations were executed successfully and that data was correctly processed and displayed. The Playwright report showed that all steps were completed without errors, indicating proper integration of the application components.

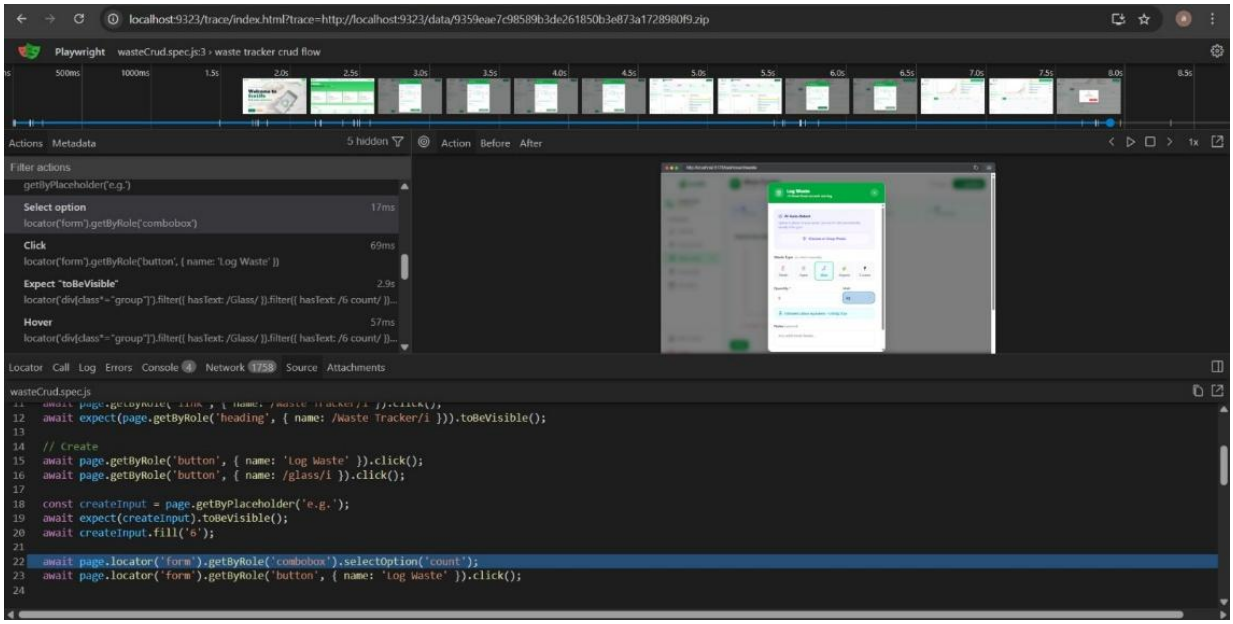




2.4.3. Waste Log CRUD

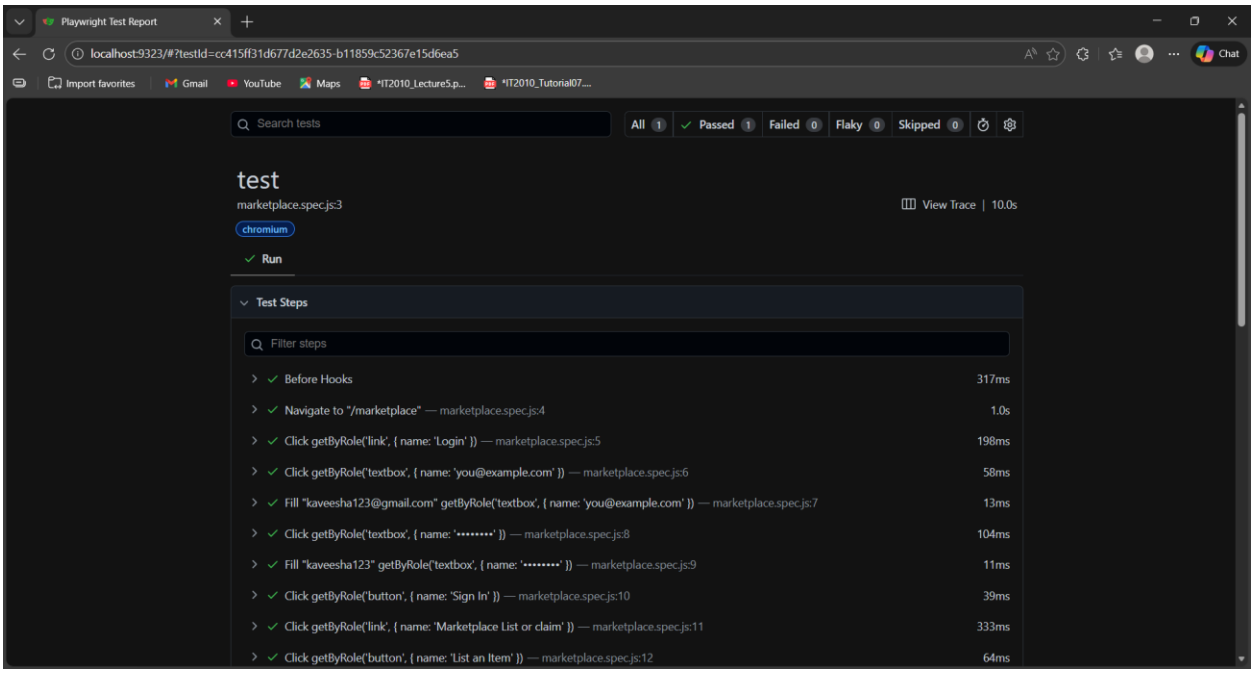
Integration testing for the Waste Tracker module was carried out by validating the complete workflow from login to waste log creation. The test confirmed that user actions were processed correctly and that data was successfully handled and displayed. The Playwright report showed that all steps were completed without errors, indicating proper integration of the application components.

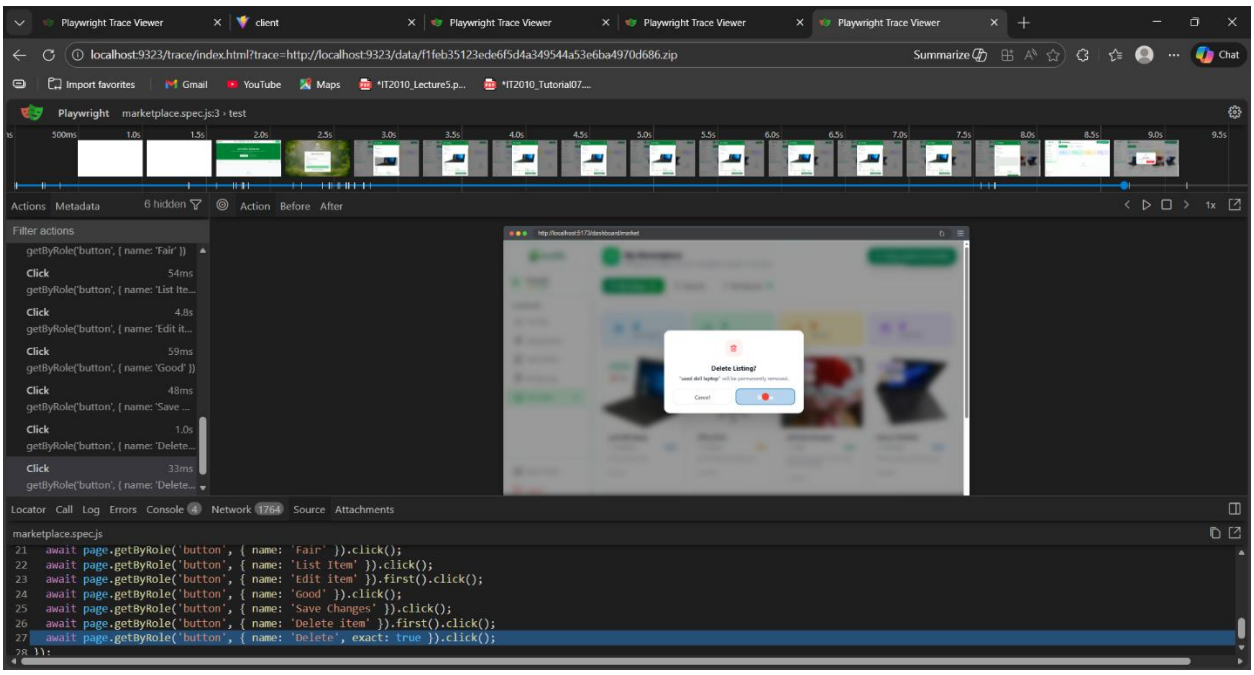




2.4.4. Marketplace CRUD

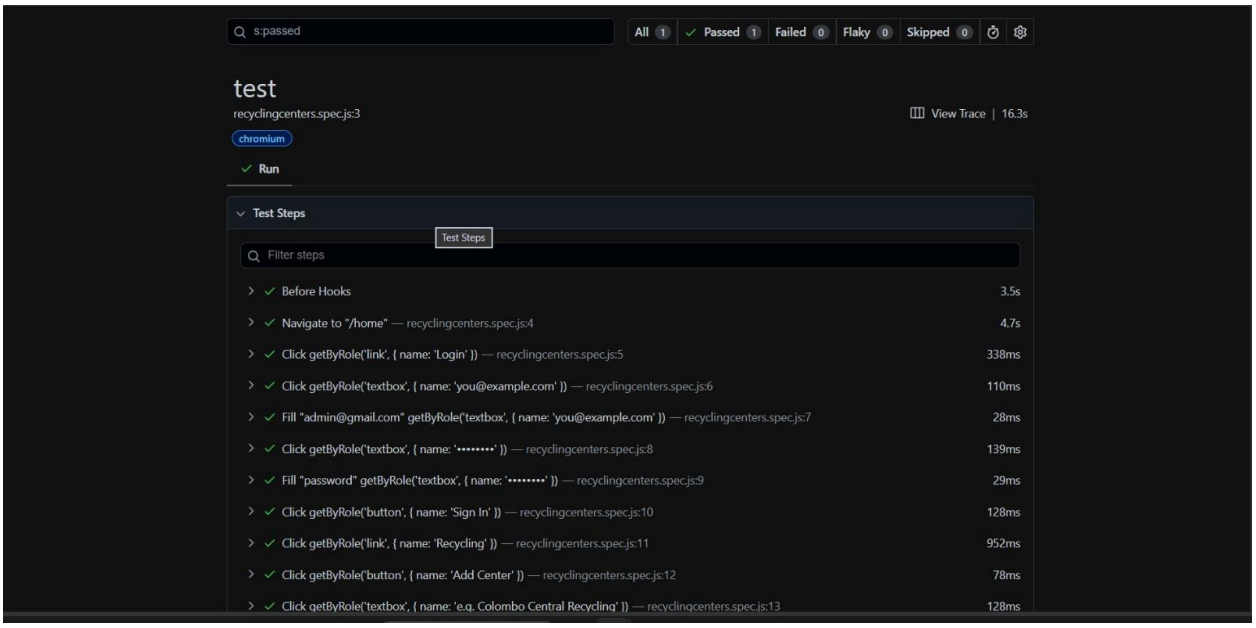
Integration testing for the Marketplace module was carried out by validating the complete workflow from login to item listing management. The test confirmed that CRUD operations were executed successfully and that data was correctly processed and displayed. The Playwright report showed that all steps were completed without errors, indicating proper integration of the application components.

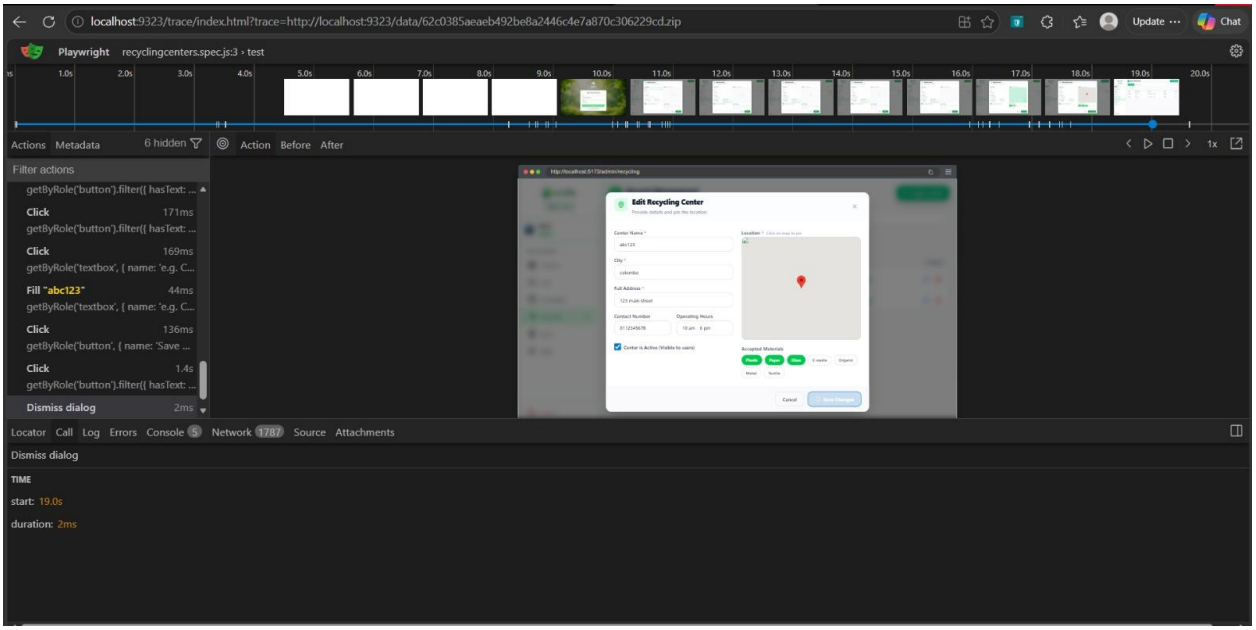




2.4.5. Recycling Module CRUD

Integration testing for the Recycling Centers module was carried out by validating the complete workflow from user authentication to recycling center management. The test confirmed that users could successfully perform CRUD operations such as creating, updating, and deleting recycling center details. It also verified that the frontend, backend API, and database interacted correctly, with all data changes being accurately processed and displayed. The Playwright report showed that all steps were completed without errors, indicating proper integration of the application components.





3. Performance testing setup and execution

3.1. Performance Testing

Performance testing refers to the process of evaluating how well the application performs under different levels of load. It is used to measure factors such as response time, system stability, and the ability to handle multiple user requests simultaneously. This type of testing ensures that the system can operate efficiently without delays or failures when used by multiple users at the same time.

3.2. Setup

Performance testing was conducted using Artillery.io, a lightweight load testing tool suitable for Node.js and Express-based applications. The backend server was hosted locally on http://localhost:5000, and test scenarios were defined using a YAML configuration file.

The test configuration included:

- A duration of 30 seconds
- A load of 5 requests per second
- Multiple API scenarios representing group modules:
    - Waste Analytics (GET /api/waste/analytics)
    - Energy Appliances (GET /api/energy/appliances)
    - Marketplace Items (GET /api/marketplace/items)
    - Recycling Centers (GET /api/recycling/centers)

Authentication was handled by including a valid JWT token in the Authorization header to simulate real-world authorized API usage.

3.3. Execution

The performance test was executed using the following command:

```
npx artillery run performance-test.yml --output report.json
```

During execution, Artillery simulated multiple virtual users sending concurrent requests to the backend API. The system processed requests continuously across all modules during the 30-second test period.

performance-test.yml U

server > performance-test.yml

```yaml
config:
  target: "http://localhost:5000"
  phases:
    - duration: 30
      arrivalRate: 5
  defaults:
    headers:
      Authorization: "Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjY5Y2ZmNTE4NmI1MmJhOTVmMjlkYjM5NSIsInJvbGUiOiJ1c2VyIiwia
      Content-Type: "application/json"

scenarios:
  - name: "Waste Analytics"
    flow:
      - get:
          url: "/api/waste/analytics"

  - name: "Energy Appliances"
    flow:
      - get:
          url: "/api/energy/appliances"

  - name: "Marketplace Items"
    flow:
      - get:
          url: "/api/marketplace/items"

  - name: "Recycling Centers"
    flow:
      - get:
          url: "/api/recycling/centers"
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
amaya@Toothless MINGW64 ~/Desktop/recycle/EcoLife/server (tests/Amaya-testcase)
$ npx artillery run performance-test.yml --output report.json
-------------------------------
Summary report @ 11:06:28(+0530)
-------------------------------

http.codes.200: ..................................................... 150
http.downloaded_bytes: .............................................. 423210
http.request_rate: .................................................. 5/sec
http.requests: ...................................................... 150
http.response_time:
  min: ............................................................... 147
  max: ............................................................... 830
  mean: ............................................................... 427.4
  median: ............................................................ 468.8
  p95: ............................................................... 685.5
  p99: ............................................................... 742.6
http.response_time.2xx:
  min: ............................................................... 147
  max: ............................................................... 830
  mean: ............................................................... 427.4
  median: ............................................................ 468.8
  p95: ............................................................... 685.5
  p99: ............................................................... 742.6
http.responses: ..................................................... 150
vusers.completed: ................................................... 150
vusers.created: ..................................................... 150
vusers.created_by_name.Energy Appliances: ........................... 32
vusers.created_by_name.Marketplace Items: ........................... 39
vusers.created_by_name.Recycling Centers: ........................... 40
vusers.created_by_name.Waste Analytics: ............................ 39
```

Figure - Artillery performance test summary showing successful handling of concurrent requests across Waste, Energy, Marketplace, and Recycling modules with stable response times.

3.4. Results and Analysis

Based on the final summary report:

- Total Requests: 150
- Request Rate: 5 requests per second
- Successful Responses: 150 (HTTP 200)
- Failed Requests: 0

Response Time Metrics:

- Average Response Time: 427.4 ms
- Maximum Response Time: 830 ms
- Median Response Time: 468.8 ms
- 95th Percentile (p95): 685.5 ms

Module-wise Request Distribution:

- Energy Appliances: 32
- Marketplace Items: 39
- Recycling Centers: 40
- Waste Analytics: 39

These results indicate that all API requests were successfully processed without any failures. Compared to previous test results, the response times have improved, with most requests completing within less than one second. This demonstrates that the system performs efficiently and remains stable under moderate load conditions across all tested modules.

3.5. Conclusion

The performance testing results show that the system can handle concurrent API requests efficiently across multiple modules. The application remained stable under load with zero failures, and response times were within acceptable limits. This confirms that the system is capable of supporting multiple users simultaneously while maintaining reliable performance.

4. Testing Environment Configuration Details

The testing activities for the EcoLife system were carried out in a local development environment. Both the frontend and backend applications were executed locally, while MongoDB was used as the database service. Playwright was used for unit-level and integration-level testing, and Artillery.io was used for performance testing.

The environment configuration used for testing included:

- **Operating System:** Windows 11
- **Frontend Framework:** React with Vite
- **Backend Framework:** Node.js with Express.js
- **Database:** MongoDB
- **Frontend URL:** http://localhost:5173
- **Backend URL:** http://localhost:5000
- **Testing Tools:** Playwright, Artillery.io
- **Authentication Setup:** Saved authentication state / JWT token for protected routes
- **Browser Used for Testing:** Chromium

Before executing the tests, the required dependencies were installed using npm install, and both frontend and backend servers were started. The database connection was configured through environment variables, and authenticated test scenarios were executed using either a saved login state or a valid token depending on the test type.

This environment ensured that the system could be tested under realistic local development conditions, covering functional validation, integration workflows, and performance behavior.

5. Overall Conclusion

The testing activities carried out for the EcoLife system confirm that the application is functioning correctly across all major modules. Unit testing ensured that individual components such as form validations and user inputs behave as expected. Integration testing verified that different parts of the system, including the frontend, backend, API endpoints, and database, work together smoothly without errors.

Performance testing results demonstrated that the system can handle multiple concurrent requests efficiently, maintaining stable response times with zero failures under moderate load conditions. This indicates that the application is reliable and capable of supporting real-world usage scenarios.

Overall, the testing process helped identify and prevent potential issues, improved system stability, and ensured data integrity across all modules. The EcoLife system is therefore considered to be stable, reliable, and ready for deployment.